

Topic: DOM Manipulation
Date: 27 Sept 2025
Mentor: Sushant Goyal

Recap:

1. HTML

- a. what are HTML tags
- b. what are semantic tags
- c. little bit about SEO
- d. Developer tools in browser

2. CSS

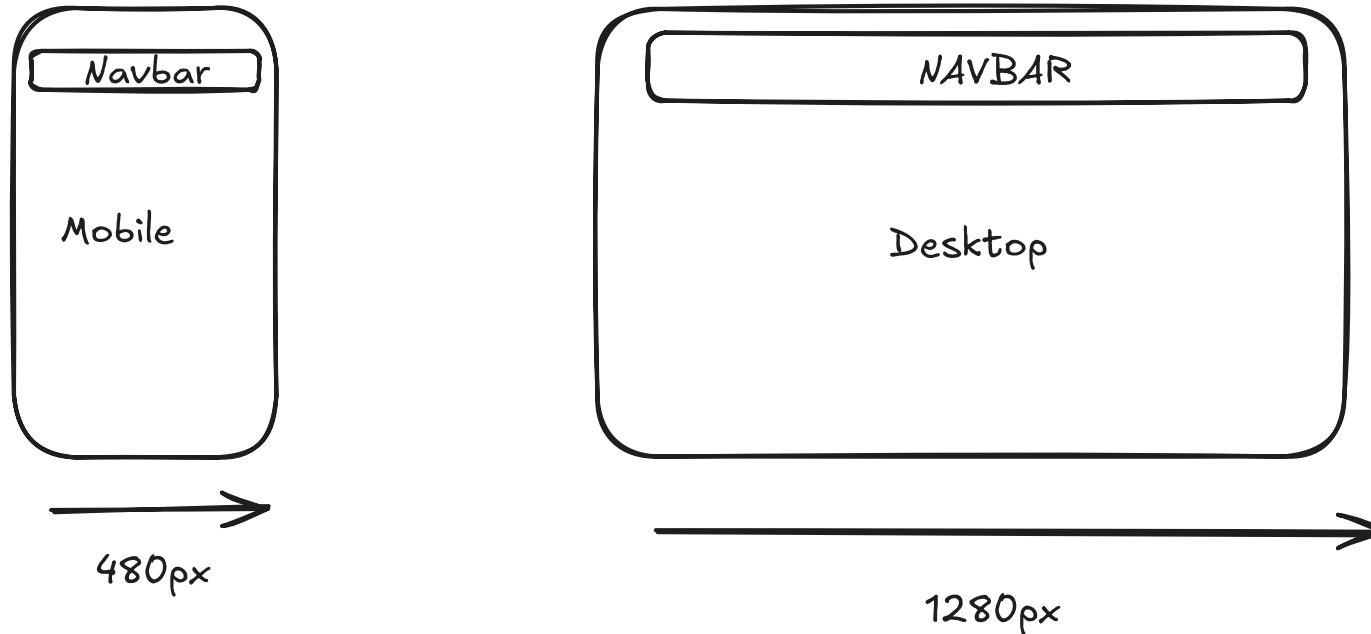
- a. using plain CSS
- b. using tailwind
- c. Order of inclusion and its importance

3. Javascript

- a. creating variables
- b. functions
- c. loops
- d. conditional statements

Breakpoints in CSS

1. they are specific points at which webpage design and layout adapt to different screen sizes



1. It will go out of the screen
 - visibility will be there but content is to be compromised
2. it will squeeze into size and try to fit in available dimensions
 - the visibility will be compromised

Here comes breakpoints into picture

- it creates a point at which the styling has to change

- these points are generic and they can work for a series of devices

General Breakpoints:

1. 480px - for mobile devices
2. 768px - for tablet devices
3. 1024px - for small screen desktop
4. 1248px - for medium screens
5. 1440px - for large screens
6. - beyond that

≤ 480 px of screen width

>480 px and ≤ 768 px

>768 px and ≤ 1024 px

HTML structure

Inclusion of CSS into HTML

How we can change CSS from the browser

How to carry that CSS into the code

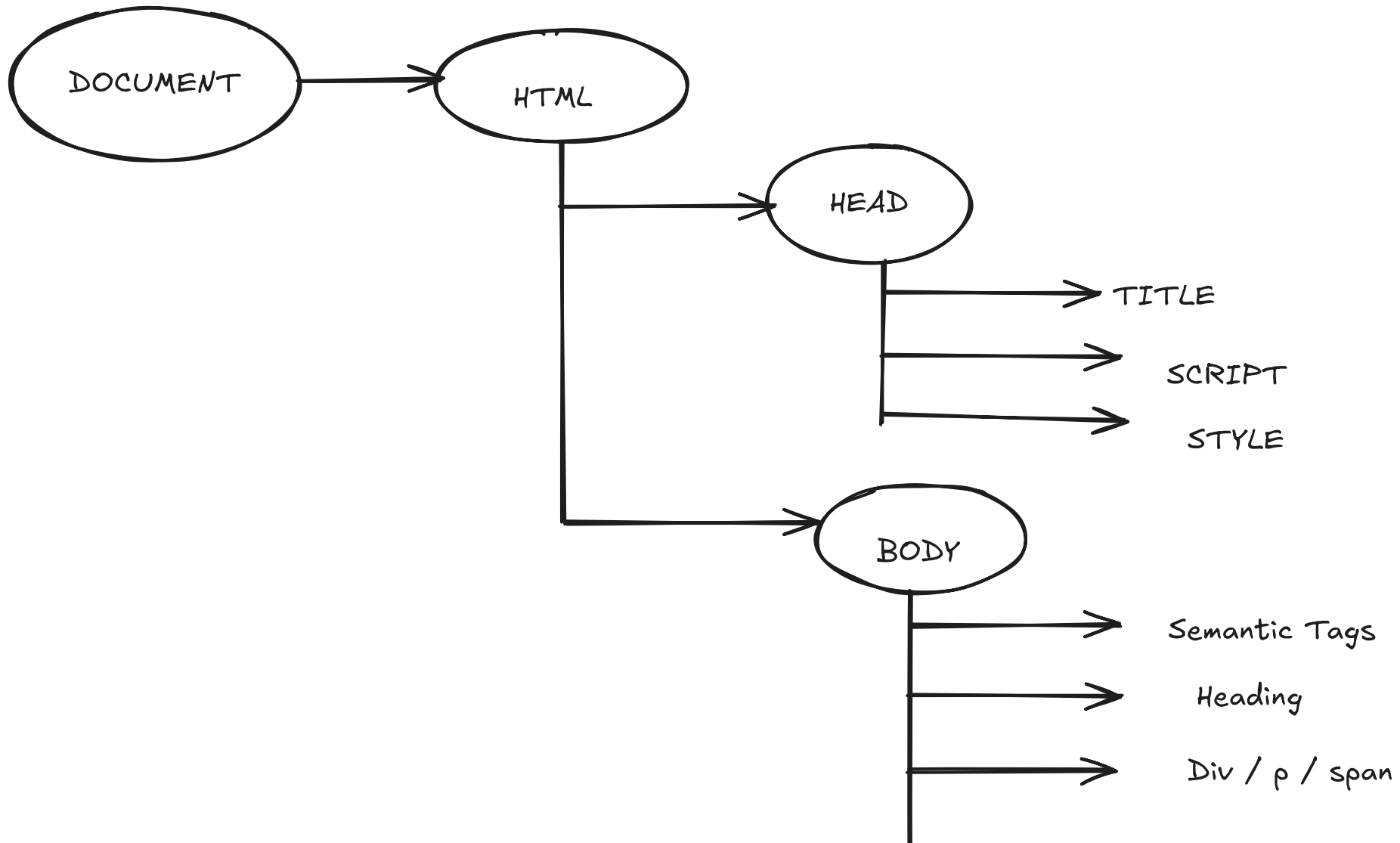
DOCTYPE - Document Type

DOM - Document Object Model

it is a programming interface for web documents

it represents the page so that programs like JS can change the document structure, style and content dynamically.

When a web page loads the browser creates a TREE like structure that mirrors HTML elements



```
var obj = {  
  key: {  
    key1: value1  
  }  
}
```

```
obj.key.key1 = 'other value'  
obj.key.key2 = 'some value'
```

Using DOM - Tree based structure:

1. Access the elements present on the page
2. Change the content - text, html
3. Change the style, attributes
4. Add, remove or reorder elements dynamically
5. listen to and react to the events (clicks, key press, input)

Nodes - Each part of DOM is a node.

- Element Nodes: corresponds to HTML tags like div, p, span
- Text Nodes: text inside the elements
- Attribute Nodes: Attributes like class and id

'DOCUMENT' is the entry point to the DOM tree.

Task: Try writing document in the console and see its output

1. Access the elements present on the page

- a. using ID
- b. using CLASS
- c. using TAG

Access elements using tags

`getElementsByTagName('tag')`

- we can traverse through the list like an array
- we can make changes to the html content

Access elements using class

`getElementsByClassName('class')`

- we can traverse and make changes
- can give multiple elements

Access elements using ID

`getElementById('id')`

- we can traverse and make changes
- will give one element only.

2. Change the content of elements

`innerText` and `innerHTML`

both of these can help in changing the content inside elements but they behave differently

`innerText`:

- Set or get the visible text content
- strips out HTML tags and show only text
- it is safer when we want plain text

`innerHTML`

- set or get the HTML content inside the element
- parses the string as HTML so that tags are rendered
- can insert complex HTML structure

3. Change the styles and attributes

Style Manipulation

`element.style.property = 'value'`

using this we can change the style for an element

if we want to override the `INLINE` style completely

`element.style.cssText = 'value in CSS format'`

`// element.style.cssText = 'color:white;background-color:blue;'`

Managing Classes

1. `add(className)` - adds a class if it doesn't exist

1. `.add(className)` - adds a class if it doesn't exist
2. `.remove(className)` - remove a class if it exists
3. `.toggle(className)` - adds if the class is missing, or removes if present
4. `.contains(className)` - check if the class exists (returning true or false)

Getting and Setting attributes

1. `getAttribute(attribute)`
get the attribute value based on the attribute name
2. `setAttribute(attribute, attrValue)`
set the value for attribute